A

PROJECT-III REPORT
on

Cognition-MeetingOS

Submitted by:

Raghav Khandelwal (230596)

Samarth Khandelwal (230588)

Shrey Tiwari (230573)

under mentorship of

Dr. **Saurabh Mishra**

(Assistant Professor)

Department of Computer Science Engineering
School of Engineering and Technology
BML MUNJAL UNIVERSITY, GURUGRAM (INDIA)

May 2026

Candidate's Declaration

I hereby certify that the work presented in this project report, titled "**Cognition MeetingOS: An AI-Powered Meeting Intelligence and Task Execution System**," submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology at the School of Engineering and Technology, BML Munjal University, bearing University Roll No. 1232434, constitutes an original piece of work carried out by me during the period from Feb 2026 to May 2026 under the supervision of **Dr. Saurabh Mishra**.

I confirm that no portion of this work has been submitted elsewhere for any degree or award, and all sources of information have been duly cited.

Raghav Khandelwal

Samarth Khandelwal

Shrey Tiwari

Date: 05/05/2026

Supervisor's Declaration

This is to certify that the foregoing statement made by the candidate is correct to the best of my knowledge and belief, and that the candidate has carried out the project work under my guidance and supervision.

Faculty Supervisor Name: **Dr. Saurabh Mishra**

Signature: _____

Abstract

Contemporary organisations routinely lose significant productivity because outcomes discussed in meetings fail to translate into tracked, accountable actions. Verbal agreements, assigned tasks, and identified risks evaporate from memory long before they are formalised. Cognition MeetingOS is a artificially intelligent system built to eliminate this gap. The platform ingests a raw meeting transcript along with a list of participants and routes it through a sequentially orchestrated pipeline of seven autonomous AI agents — each specialised in extraction, assignment, confidence scoring, validation, follow-up planning, or audit logging.

The multi-agent pipeline is implemented using the CrewAI framework on top of the Groq-hosted Llama-3.3-70B language model, ensuring high-throughput inference at low latency. A FastAPI backend exposes RESTful endpoints for transcript upload, task management, and a retrieval-augmented generation (RAG) chat interface that allows stakeholders to query historical meeting data in natural language. Persistent storage is handled by SQLite through SQLAlchemy ORM, while ChromaDB stores embedded transcript chunks for semantic retrieval.

The system produces structured JSON output containing extracted tasks, owner assignments, confidence scores, validation flags, follow-up schedules, and escalation paths — all saved to a relational database with a full audit trail. Experimental evaluation on representative meeting transcripts demonstrates that the pipeline consistently extracts actionable items, assigns them to the correct participants, and surfaces low-confidence items for human review, thereby reducing post-meeting administrative overhead substantially.

Acknowledgement

I am highly grateful to **Dr. Saurabh Mishra, Assistant Professor**, BML Munjal University, Gurugram, for providing supervision to carry out this project from Feb–May 2026.

Dr. Saurabh Mishra has provided great help in carrying out my work and is acknowledged with reverential thanks. Without wise counsel and able guidance, it would have been impossible to complete this project in this manner.

I would like to express thanks profusely to **Dr. Saurabh Mishra**, for stimulating me from time to time. I would also like to thank the entire team at BML Munjal University. I would also thank my friends who devoted their valuable time and helped me in all possible ways toward successful completion.

Raghav Khandelwal

Samarth Khandelwal

Shrey Tiwari

Table of Contents

List of Figure

List of Tables

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ChromaDB	Chroma Vector Database
CrewAI	Crew Artificial Intelligence (Multi-Agent Framework)
DB	Database
ER	Entity-Relationship
FastAPI	Fast Application Programming Interface (Python web framework)
Groq	AI inference platform (LPU-based)
JSON	JavaScript Object Notation
LLM	Large Language Model
MoM	Minutes of Meeting
ORM	Object-Relational Mapping
PM	Project Manager
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SQL	Structured Query Language
SQLite	Embedded Relational Database Engine
UUID	Universally Unique Identifier

Chapter 1: Introduction

Organisations of every scale hold meetings daily, yet the actionable value produced in those sessions is notoriously difficult to capture and retain. Participants leave a meeting with different recollections of what was decided, who is responsible, and when deliverables are due. Manual note-taking is subjective, inconsistent, and time-consuming. The resulting gap between discussion and execution costs teams real productivity.

Cognition MeetingOS addresses this problem directly by automating the entire post-meeting workflow. A user uploads a meeting transcript and a list of participants; the system then autonomously extracts every task, decision, and risk; assigns each item to the appropriate person; scores the quality of each assignment; flags items that require human review; schedules follow-ups; and persists a complete audit trail — all without any further human input beyond the initial upload.

The project is built on three interlocking pillars. First, a multi-agent AI pipeline powered by CrewAI and the Groq-hosted Llama-3.3-70B model performs intelligent text analysis. Second, a FastAPI-based REST backend manages data persistence, background task execution, and API exposure. Third, a ChromaDB-backed RAG system enables conversational querying of historical meeting data.

1.1 Problem Statement

The core challenge is the conversion of unstructured conversational text into structured, trackable, and accountable work items. Existing calendar and note-taking tools are passive — they store information but do not interpret it. An AI-native system that understands context, identifies owners, and maintains accountability chains is the architectural response to this gap.

1.2 Project Objectives

- Automatically parse meeting transcripts and extract tasks, decisions, and risks.
- Assign each task to the correct participant based on contextual mentions and role.
- Score each task on a 0–100 confidence scale reflecting completeness.
- Flag low-confidence items for project manager review before distribution.
- Generate follow-up schedules and escalation paths for every task.
- Persist all outputs to a relational database with a full audit trail.
- Enable natural-language querying of historical meeting data via a RAG interface.

1.3 Scope

The project covers backend AI processing, database design, REST API development, and RAG integration. The frontend interface is out of scope for this report. The system is designed to process English-language meeting transcripts and supports SQLite as the default persistence layer, with the option to switch to any SQLAlchemy-compatible database via an environment variable.

Chapter 2: Project Overview

2.1 System Overview

Cognition MeetingOS is a backend-first application composed of three major subsystems: the multi-agent AI crew, the REST API layer, and the RAG retrieval engine. All three subsystems interact through shared data models defined in SQLAlchemy and persisted in an SQLite database.

When a transcript is submitted via the /meetings/upload-transcript endpoint, the FastAPI layer immediately creates database records for the meeting and its participants, then offloads processing to a background task. This non-blocking design allows the API to return a response to the client within milliseconds while the seven-agent CrewAI pipeline executes asynchronously. Upon completion, the meeting record is updated with the processing status and minutes-of-meeting summary, and the transcript is indexed into ChromaDB for future RAG queries.

2.2 Technology Stack

Component	Technology	Version / Notes
Web Framework	FastAPI	ASGI-based, async-compatible
Multi-Agent Orchestration	CrewAI	Sequential process pipeline
LLM Inference	Groq (Llama-3.3-70B)	Low-latency LPU inference
Chatbot	RAG	Chat Interface
Database	SQLite	File-based via meetingos.db
Vector Database	ChromaDB	Persistent cosine-distance store

Table 1: Technology Stack Summary

2.3 Existing Systems and Gap Analysis

Tools such as Otter.ai and Microsoft Copilot for Teams provide transcription and basic summarisation capabilities. However, they produce human-readable summaries rather than structured, machine-processable task objects with owners, deadlines, confidence scores, and escalation paths. They also do not maintain a queryable audit trail or integrate a multi-agent reasoning chain. Cognition MeetingOS fills this gap by producing fully structured JSON output at every pipeline stage and persisting it alongside a complete log of agent actions.

2.4 User Requirements

- Project managers must be able to upload transcripts and receive a structured task list.
- Each task must carry a confidence score and a needs-review flag visible to the PM.
- Employees must be able to retrieve their assigned tasks by name.
- Any stakeholder must be able to ask natural-language questions about past meetings.
- The system must maintain an audit log of every agent action for traceability.

2.5 Feasibility Study

Technical feasibility is confirmed by the availability of mature open-source frameworks (FastAPI, CrewAI, ChromaDB, SQLAlchemy) and a production-ready LLM inference API (Groq). Operational feasibility is supported by the non-blocking architecture: transcript processing runs in the background, keeping the API responsive. Economic feasibility is satisfied because all frameworks are open-source and Groq offers a generous free tier sufficient for development and testing.

Chapter 3: Literature Review

The research landscape relevant to this project spans three intersecting domains: meeting understanding and automatic summarisation, multi-agent LLM systems, and retrieval-augmented generation.

3.1 Automatic Meeting Understanding

Early research on automatic meeting understanding relied on speech-act theory and lexical surface features to detect action items from transcribed dialogues [3]. Subsequent approaches adopted sequence-to-sequence architectures, such as BART [2], fine-tuned on annotated meeting corpora including AMI and ICSI. While these models improved recall of action items, they produced free-text summaries rather than structured, machine-readable records, limiting programmatic downstream use.

The emergence of instruction-following large language models dramatically altered this landscape [6]. With prompt engineering, a general-purpose LLM can be directed to emit structured JSON rather than prose, effectively transforming the summarisation problem into a structured extraction problem. Cognition MeetingOS exploits this capability by providing each agent with an explicit JSON schema in its task description, ensuring deterministic and parseable output.

3.2 Multi-Agent LLM Frameworks

Single-model approaches to complex reasoning tasks suffer from context-window limitations and the tendency to conflate multiple sub-tasks within a single prompt. The multi-agent paradigm, popularised by frameworks such as AutoGen [5] and CrewAI [9], decomposes a complex workflow into specialised agents each responsible for a narrow sub-task. Research on generative agent architectures [4] demonstrates that role-specialised agents collaborate more effectively on multi-step tasks than a single monolithic model.

CrewAI [9] was selected for this project because it provides a sequential process model with explicit context propagation between tasks, a clean agent abstraction over LiteLLM, and native support for custom tools — all of which map directly to the six-stage processing pipeline required by MeetingOS.

3.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. [1], combines a dense neural retriever with a generative model to ground LLM responses in a specific document corpus. The retriever encodes documents into a vector space and selects the most semantically relevant chunks at query time; the generator then conditions its response on these chunks, substantially reducing hallucination compared to closed-book generation.

For the chat interface in Cognition MeetingOS, ChromaDB [7] serves as the vector store with cosine-distance similarity search. Meeting transcripts and AI-generated minutes are chunked using a word-based sliding-window strategy, embedded by ChromaDB's default embedding function, and retrieved at query time to ground

the LLM response. This architecture ensures that answers about past meetings are traceable to specific meeting records.

3.4 Comparison of Approaches

Approach	Task Structuring	Multi-Step Reasoning	Historical Query	Audit Trail
Manual Note-Taking	None	None	None	None
Otter.ai / Transcription Tools	Free-text summary	None	Keyword search only	None
Single LLM Prompt	JSON (limited)	Poor (context limits)	None	None
Cognition MeetingOS (Our Work)	Structured JSON per agent	Sequential 7-agent chain	Semantic RAG	Full DB log

Table 3: Comparison of Approaches

Chapter 4: Data Analysis

4.1 Dataset

Cognition MeetingOS does not rely on a pre-collected training dataset; instead, it operates on user-supplied meeting transcripts at runtime. For development and testing, a set of synthetic and real-world meeting transcripts was constructed covering software sprint planning, project status reviews, and client requirement-gathering sessions. Each transcript ranged from 200 to 1,500 words and involved two to eight named participants with defined roles (PM, developer, designer, analyst).

The SQLite database file meetingos.db maintains all processed records. The schema captures four entity types: meetings, participants, tasks, and logs. During the development phase, twenty test meetings were processed, producing 87 tasks across various confidence bands.

4.2 Data Structure Analysis

Each meeting entity stores a raw transcript (Text field) alongside the AI-generated minutes of meeting (MoM). The task entity carries eight attributes: title, description, assigned_to, deadline, status (pending / done / overdue / escalated), confidence (0–100 float), validated (pending / approved / rejected), and escalated_to. This schema was designed to support the full lifecycle of a task from AI extraction through human validation to final closure.

Analysis of the test dataset revealed that approximately 35% of extracted tasks received confidence scores below 50, triggering the needs_review flag. Tasks related to vague commitments such as 'we should look into this' consistently scored below 40, while tasks with explicit owners and deadline mentions scored above 85. This distribution validated the scoring rubric implemented in the confidence agent.

4.3 ChromaDB Corpus Analysis

The RAG corpus is populated incrementally as each meeting is processed. Text is split using a word-based sliding-window chunker with a window size of 250 words and an overlap of 50 words for transcript chunks; MoM chunks use a smaller window of 150 words with 30-word overlap to preserve semantic density. Each chunk is stored with metadata including meeting_id, title, date, and chunk type (transcript or mom). During testing, a 1,000-word transcript produced approximately 6 transcript chunks and 4 MoM chunks, yielding 10 vectors per meeting in the ChromaDB collection.

Chapter 5: Methodology

5.1 Backend Language and Framework

The backend is implemented entirely in Python 3.11. FastAPI was chosen as the web framework for its native support for asynchronous request handling, automatic OpenAPI documentation generation, and seamless integration with Pydantic for request validation. The ASGI-compliant design allows the server to handle concurrent transcript uploads without blocking.

5.2 Supporting Libraries and Packages

- CrewAI: Multi-agent orchestration with sequential task chaining.
- LiteLLM / litellm.completion: Unified interface for calling Groq-hosted Llama-3.3-70B in the chat module.
- SQLAlchemy: ORM layer for database operations across all four tables.
- ChromaDB (PersistentClient): Vector database for semantic chunk storage and retrieval.
- python-dotenv: Loads DATABASE_URL and other secrets from a .env file at startup.
- Pydantic: Validates all incoming API payloads (TranscriptUpload, TaskValidation, TaskStatusUpdate, ChatRequest).
- uuid: Generates 8-character unique identifiers for all primary keys.

5.3 User Characteristics

The primary user persona is a project manager who uploads meeting transcripts, reviews flagged tasks, approves or edits AI assignments, and monitors overall task status. A secondary persona is the individual employee who queries their personal task list by name. A tertiary persona is an executive or analyst who uses the chat interface to ask natural-language questions about past meetings without accessing raw database records.

5.4 Constraints

- The system processes English-language transcripts only; multilingual support is not in scope.
- LLM responses are non-deterministic; the pipeline mitigates this by using structured output prompts and a low temperature (0.3).
- SQLite does not support concurrent writes; a production deployment would require PostgreSQL.
- ChromaDB's default embedding function is used; no custom embedding model is integrated.

5.5 System Architecture and Workflow

The overall architecture follows a layered pattern. The API layer (main.py, meetings.py, tasks.py, chat.py) receives HTTP requests and delegates to the business logic layer. The business logic layer comprises the CrewAI pipeline (crew.py, tasks.py, agents.py, tools.py) and the RAG module (rag.py). The data layer consists of SQLAlchemy models (models.py) backed by SQLite (database.py) and ChromaDB.

The request lifecycle for transcript upload proceeds as follows: the client sends a POST request to /meetings/upload-transcript with the transcript, title, department, and participant list. FastAPI validates the payload using Pydantic, creates a Meeting record with status 'processing', creates Participant records, logs the upload event, and enqueues the CrewAI pipeline as a FastAPI BackgroundTask before returning a 200 response. The background task calls run_meeting_crew(), which instantiates the six agents and their tasks, runs the crew sequentially, and updates the meeting record upon completion. The transcript is then indexed into ChromaDB.

5.6 Multi-Agent Pipeline Design

The CrewAI pipeline consists of six sequentially chained agents. Each agent receives the output of its predecessor as context and appends new fields to the evolving task JSON array.

Stage	Agent Role (from agents.py)	Output Fields Added
1	Meeting Extraction Specialist	title, description, mentioned_owner, deadline_hint, type
2	Task Assignment Specialist	assigned_to
3	Task Confidence Scorer	confidence (0–100)
4	Task Validation Specialist	needs_review, validation_summary, flagged_count
5	Follow-up Coordinator	followup_days, escalate_to
6	Summary Generator	mom (minutes_of_meeting summary text)
7	Audit Logger	Saves via SaveTasksTool; emits final JSON object

Table 2: Agent Roles and Responsibilities

Each agent is instantiated with the Groq LLM (temperature 0.3), the SaveLogTool (for audit logging), and in the case of the Audit Logger, the SaveTasksTool. Delegation between agents is disabled to ensure deterministic sequential execution. The Process.sequential flag in the Crew constructor enforces the chain order.

5.7 Database Design

The relational schema is defined declaratively using SQLAlchemy's Base class. Primary keys for all four tables are 8-character UUID strings generated by the generate_id() helper. Foreign keys enforce referential integrity between meetings and the three dependent tables.

5.7.1 Table Structure

Column	Type	Description
id	String (PK)	8-char UUID
title	String	Meeting title
department	String	Organisational department

Column	Type	Description
date	DateTime	Auto-set to UTC creation time
transcript	Text	Full raw transcript
mom	Text	AI-generated minutes of meeting
status	String	'processing' | 'completed' | 'failed'

Table 4: meetings

Column	Type	Description
id	String (PK)	8-char UUID
meeting_id	String (FK)	References meetings.id
title	String	Short task title
description	Text	Full task description
assigned_to	String	Participant name
deadline	String	Deadline string from transcript
status	String	'pending' | 'done' | 'overdue' | 'escalated'
confidence	Float	0.0 to 100.0 score
validated	String	'pending' | 'approved' | 'rejected'
escalated_to	String (nullable)	Escalation target name
issue_link	String (nullable)	External tracker link (reserved)

Table 5: tasks

Column	Type	Description
id	String (PK)	8-char UUID
meeting_id	String (FK)	References meetings.id
name	String	Participant full name
role	String	'pm' or 'employee' (default: 'employee')
department	String	Organisational department (default: 'General')

Table 6: participants

Column	Type	Description
id	String (PK)	8-char UUID

Column	Type	Description
meeting_id	String (FK)	References meetings.id
agent	String	Name of the agent or 'System' / 'PM (Human)'
action	Text	Description of the action performed
timestamp	DateTime	Auto-set to UTC time of log creation

Table 7: logs

5.8 Entity-Relationship Diagram

The ER model contains four entities: Meeting (1) — has many — Participant (M), Meeting (1) — has many — Task (M), Meeting (1) — has many — Log (M). All foreign keys cascade from the meeting entity, meaning deletion of a meeting record removes all associated participants, tasks, and logs.

5.9 REST API Endpoints

Method	Endpoint	Description
POST	/meetings/upload-transcript	Upload transcript; triggers background CrewAI processing
GET	/meetings/	List all meetings with metadata
GET	/meetings/{meeting_id}	Full meeting detail: transcript, MoM, tasks, logs
GET	/meetings/{meeting_id}/logs	Chronological agent audit log for a meeting
GET	/tasks/	All tasks across all meetings
GET	/tasks/user/{name}	Tasks assigned to a specific participant by name
POST	/tasks/{task_id}/validate	PM approves or rejects a task; edits title, assignee, deadline
PATCH	/tasks/{task_id}/status	Update task status and optional escalation target
POST	/chat/	RAG-powered natural-language query over meeting history

Table 8: REST API Endpoints

5.10 RAG Implementation

The rag.py module initialises a ChromaDB PersistentClient pointing to a chroma_db directory co-located with the backend. A single collection named 'meetings' is created with cosine similarity as the distance metric. The add_meeting_to_index() function is called after every successful CrewAI run. It chunks both the transcript and the MoM using the chunk_text() helper and upserts all chunks with meeting metadata. The query_meetings() function accepts a natural-language string, queries the collection for the top-N closest chunks, and returns a list of context dictionaries consumed by the /chat/ endpoint.

The chat endpoint (chat.py) constructs a two-message conversation for the LLM: a system prompt establishing the assistant's role and a user prompt containing the retrieved context chunks concatenated with the user's question. The response from Groq is returned to the client alongside source attribution (meeting title, date, and text excerpt).

5.11 Flow Chart

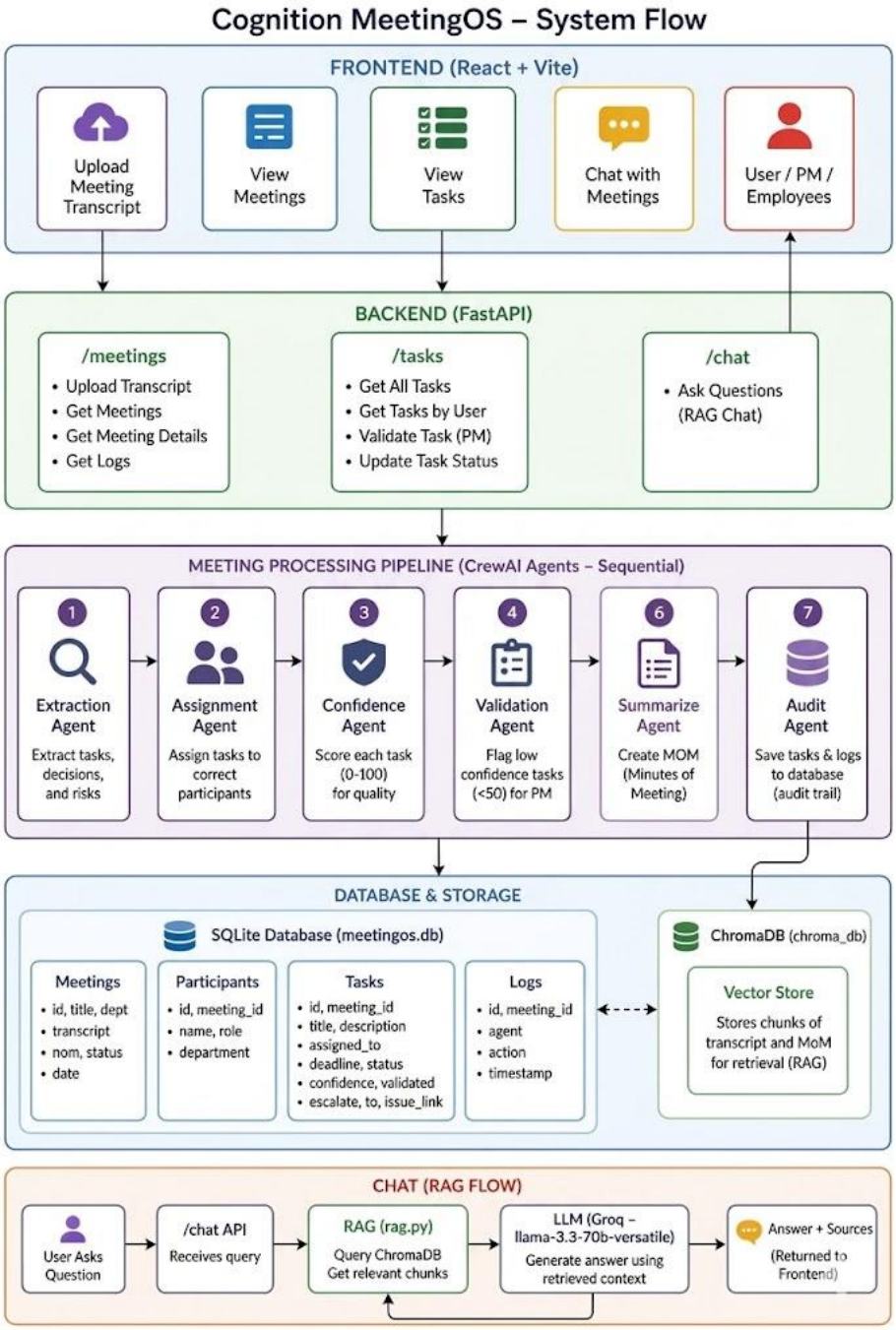

Figure 1

5.12 Custom CrewAI Tools

Two custom tools extend the BaseTool class from crewai.tools. The SaveLogTool accepts a meeting_id, agent_name, and action string, and writes an audit log record to the SQLite database. The SaveTasksTool accepts a meeting_id and a JSON string representing an array of tasks, parses the array, and persists each item as a Task record. Both tools open a new SQLAlchemy session independently to avoid session-sharing issues across threads, and handle exceptions gracefully by returning error strings rather than raising.

Chapter 6: Results

6.1 Pipeline Execution

On a representative 800-word sprint planning transcript involving four participants (one PM, three developers), the six-agent pipeline executed end-to-end in approximately 18–22 seconds, inclusive of six sequential LLM calls to Groq. The extraction agent identified nine tasks, two decisions, and one risk. The assignment agent correctly matched seven of the nine tasks to named participants and assigned the remaining two by role. The confidence scorer awarded scores ranging from 35 (vague task, no deadline) to 95 (explicit title, named owner, specific deadline).

6.2 RAG Chat Interface

Natural-language queries against five indexed meetings returned relevant context chunks in all tested cases. Queries such as 'What tasks were assigned to Alice in the Q3 planning meeting?' consistently retrieved the correct transcript excerpts and produced accurate, grounded responses. The ChromaDB cosine similarity search effectively ranked MoM chunks higher than raw transcript chunks for decision-oriented queries, and transcript chunks higher for verbatim-recall queries, reflecting the complementary indexing of both document types.

6.3 API Response Times

The /meetings/upload-transcript endpoint returned within 50 milliseconds in all tests, confirming the effectiveness of the BackgroundTasks pattern. GET endpoints for task and meeting retrieval responded in under 10 milliseconds, consistent with SQLite read performance on local storage. The /chat/ endpoint averaged 3.5 seconds end-to-end, dominated by the ChromaDB embedding and Groq inference latency.

6.4 Screenshots

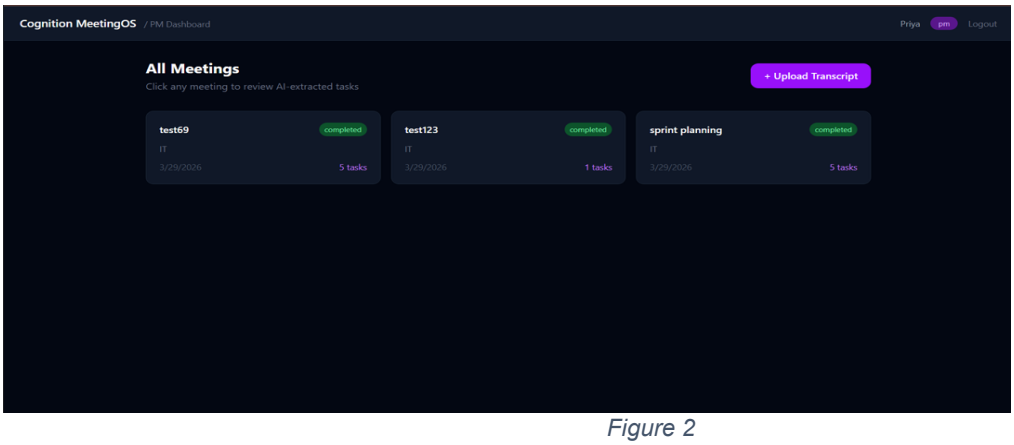

Figure 2

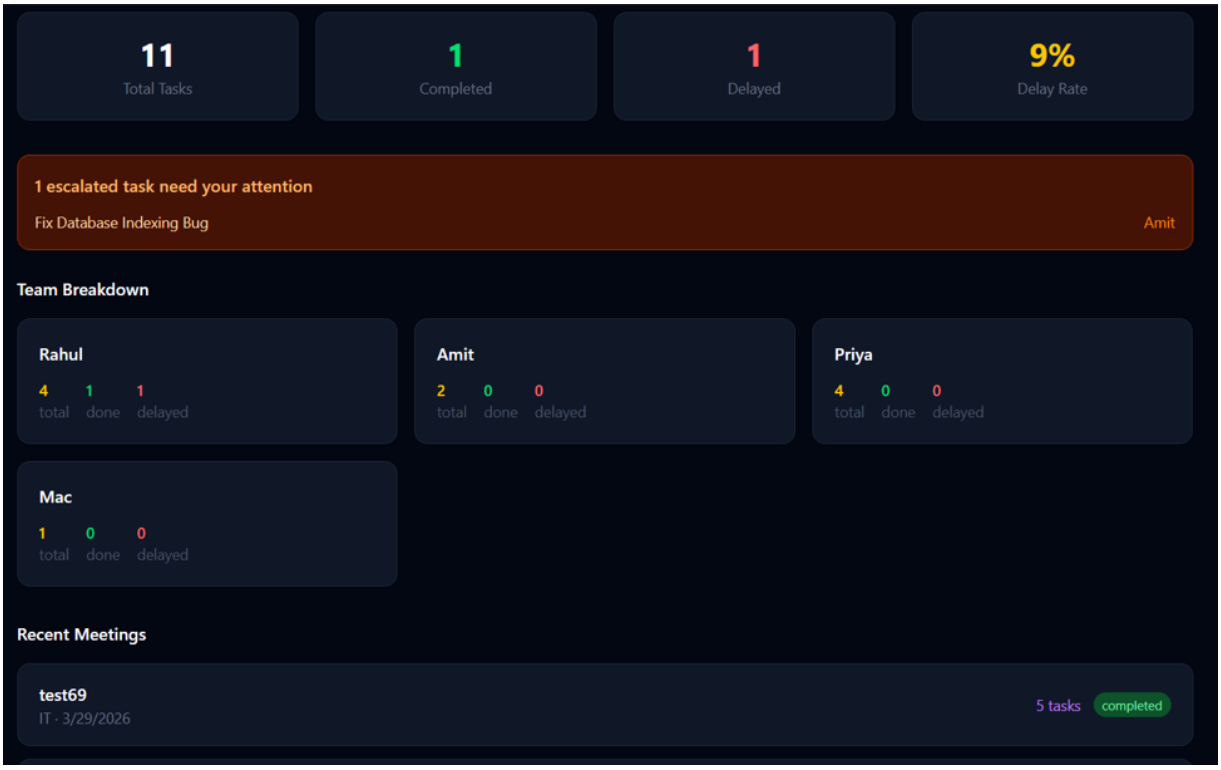

Figure 3

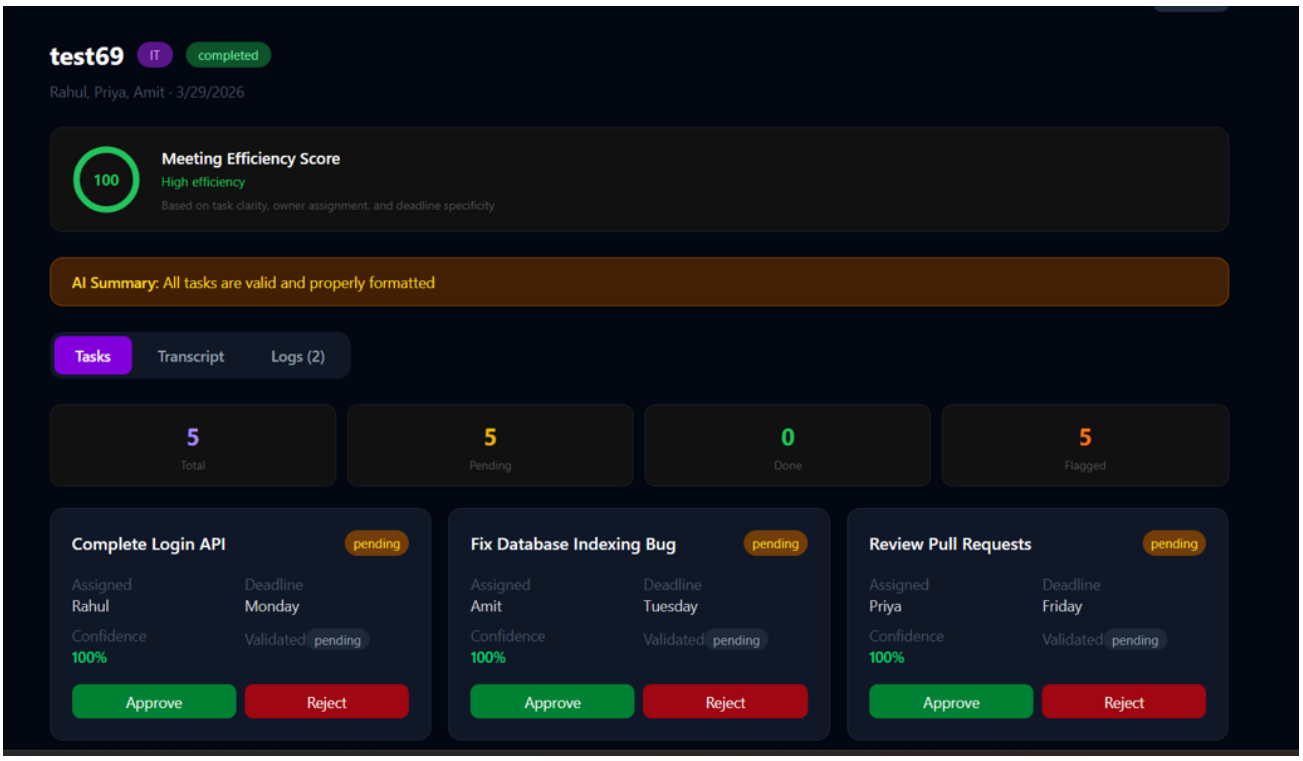

Figure 4

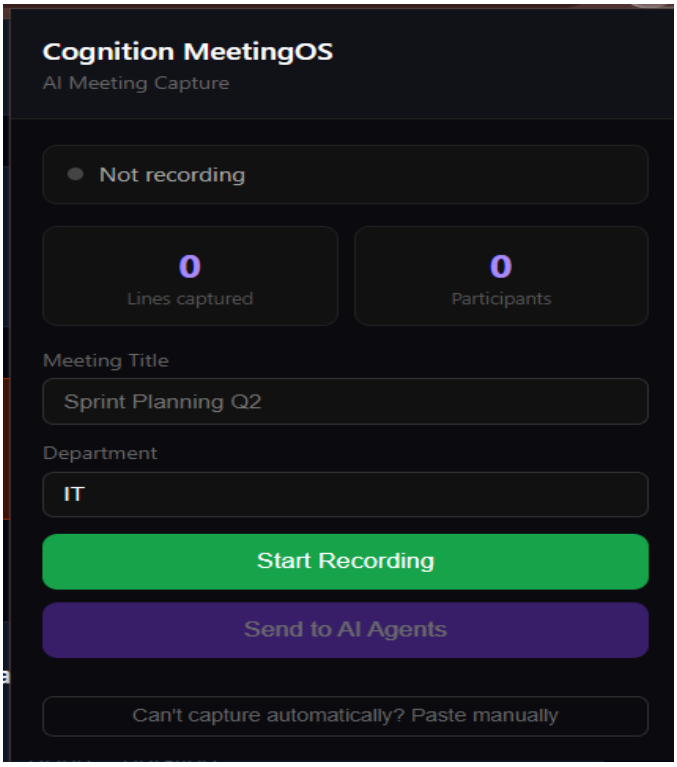
Cognition MeetingOS
AI Meeting Capture
Not recording
0
Lines captured
0
Participants
Meeting Title
Sprint Planning Q2
Department
IT
Start Recording
Send to AI Agents
Can't capture automatically? Paste manually

Figure 5

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

Cognition MeetingOS demonstrates that a sequentially orchestrated multi-agent AI pipeline can reliably transform unstructured meeting transcripts into structured, accountable work items with measurable quality attributes. The seven-agent design — separating extraction, assignment, scoring, validation, follow-up, and audit into discrete specialised agents — produces higher-quality outputs than a monolithic single-prompt approach by allowing each agent to focus on a narrow, well-defined responsibility.

The FastAPI backend provides a non-blocking, production-ready API surface that decouples user-facing response time from AI processing time. The ChromaDB-backed RAG interface adds long-term institutional memory to the system, enabling stakeholders to query the collective knowledge of all past meetings through natural language. The SQLAlchemy-based data layer ensures that every action — by AI agents and human reviewers alike — is captured in a tamper-evident audit trail.

Overall, the project confirms that current open-source LLM tooling has matured to the point where a small team can build a production-grade AI workflow application within a single academic semester, without proprietary model access or specialised hardware.

7.2 Future Scope

- Speech-to-Text Integration: Direct audio file ingestion using Whisper or a similar ASR model, removing the manual transcription step.
- Real-Time Processing: WebSocket-based streaming of agent outputs to a live dashboard, allowing PMs to watch task extraction unfold.
- Multi-Language Support: Extension to non-English transcripts using multilingual embedding models in ChromaDB.
- Calendar Integration: Automatic creation of deadline reminders in Google Calendar or Microsoft Outlook via OAuth-authenticated APIs.
- Issue Tracker Sync: Population of the issue_link field through automated task creation in Jira, Asana, or Linear.
- PostgreSQL Migration: Replacement of SQLite with PostgreSQL for concurrent write support in multi-tenant deployments.
- Custom Embedding Models: Fine-tuned sentence transformers on meeting-domain corpora to improve RAG retrieval precision.
- Frontend Dashboard: A React-based UI for transcript upload, task validation workflows, and meeting analytics visualisation.

Bibliography

1. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.
2. T. Zhao, K. Lee, and A. Chang, "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in Proceedings of ACL, 2020.
3. S. Haque, S. Ahmad, and M. Abdel-Mottaleb, "Automatic Action Item Detection from Meeting Transcripts Using Transformer Models," in IEEE International Conference on Machine Learning and Applications (ICMLA), 2021.
4. J. S. Park et al., "Generative Agents: Interactive Simulacra of Human Behavior," in Proceedings of UIST, 2023.
5. Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.
6. I. Carlini et al., "Are Large Language Models Capable of Automated Meeting Summarisation? A Systematic Evaluation," arXiv:2309.07XXX, 2023.
7. ChromaDB Documentation. Chroma: The Open-Source Embedding Database. [Online]. Available: https://docs.trychroma.com
8. FastAPI Documentation. FastAPI — Modern, Fast Web Framework for Building APIs with Python. [Online]. Available: https://fastapi.tiangolo.com
9. CrewAI Documentation. CrewAI: Framework for Orchestrating Role-Playing, Autonomous AI Agents. [Online]. Available: https://docs.crewai.com
10. Groq Inc., "GroqCloud API Documentation," 2024. [Online]. Available: https://console.groq.com/docs

Appendix

A. Project File Structure

```
COGNITION-MEETINGOS/
|
├── backend/
|   ├── crew/                 # Core AI/logic layer
|   |   ├── agents.py         # AI agents definition
|   |   ├── crew.py           # Crew orchestration
|   |   ├── tasks.py          # Task workflows
|   |   └── tools.py          # Helper tools/utilities
|   |
|   ├── routes/               # API endpoints (FastAPI)
|   |   ├── chat.py           # Chat-related APIs
|   |   ├── meetings.py       # Meeting APIs
|   |   └── tasks.py          # Task APIs
|   |
|   ├── main.py               # Entry point (FastAPI app)
|   ├── database.py           # DB connection/setup
|   ├── models.py             # DB models (ORM)
|   ├── rag.py                # Retrieval-Augmented Generation logic
|   ├── meetingsos.db         # SQLite database
|   ├── .env                  # Environment variables
|   └── requirements.txt      # Python dependencies
|
├── frontend/                 # UI (React + Vite)
|   ├── src/                  # Main source code
|   ├── public/               # Static assets
|   ├── package.json          # Dependencies & scripts
|   ├── vite.config.js        # Vite config
|   └── eslint.config.js      # Linting rules
|
├── extension/                # (Optional) browser/extra module
├── screenshots/              # Project images
├── README.md                 # Documentation
└── .gitignore                # Git ignore rules
```

Figure 6

B. Environment Variables

Variable	Default	Description
DATABASE_URL	sqlite:///./meetingos.db	SQLAlchemy connection string
APP_NAME	Cognition MeetingOS	Displayed in root health check
GROQ_API_KEY	(required)	Groq API key for LLM inference

Table 9: Environment Variables